EXPERIMENT 3

To Deploy Multi-Service Applications Using Docker Compose

Name	Khan Ashhad
Roll Number	2401048
Class / Batch	T1/T12

1. Aim

To deploy and manage a multi-service application using Docker Compose by defining, building, and running multiple interconnected containers through a single YAML configuration file.

2. Theory

Docker is a platform that packages applications and their dependencies into containers so they can run consistently across different environments. A container is an isolated runtime instance created from an image.

Docker Compose is a tool for defining and running multi-container applications. Services, networks, ports, environment variables, health checks, and dependencies can be described in a compose.yaml file. A single command can then build and start the complete application.

In this experiment, a Python Flask application provides a webpage, while Redis stores a visit counter in memory. The web service communicates with Redis over the private network created by Docker Compose. The browser accesses Flask through host port 5000.

Important Components

- Docker image: A template used to create containers.
- Container: An isolated environment in which an application runs.
- Dockerfile: Contains instructions to build the Flask application image.
- compose.yaml: Defines and configures the web and Redis services.
- Port mapping: 5000:5000 maps host port 5000 to container port 5000.
- Service discovery: The web service reaches Redis by the service name 'redis' on internal port 6379.
- Health check: Redis is checked with redis-cli ping before the web service starts.
- Restart policy: unless-stopped allows a service to restart unless it has been deliberately stopped.

3. Requirements

- Windows computer with Docker Desktop and Docker Engine running.
- Docker Compose plugin.
- Visual Studio Code or another text editor.
- Web browser and internet connection for downloading the required images.

4. Project Structure

```
docker-compose-experiment/
├── app.py
├── requirements.txt
├── Dockerfile
└── compose.yaml
```

5. Program Files

5.1 app.py

```
import os
import redis
from flask import Flask

app = Flask(__name__)

cache = redis.Redis(
    host=os.getenv("REDIS_HOST", "redis"),
    port=6379,
    decode_responses=True,
    socket_connect_timeout=2
)

@app.route("/")
def home():
    try:
        visits = cache.incr("visits")
        status = "Connected"
    except redis.RedisError:
        visits = "Unavailable"
        status = "Disconnected"

    return f"""
    <html>
    <head><title>Docker Compose Experiment</title></head>
    <body style="font-family:Arial;text-align:center;margin-top:80px">
        <h1>Docker Compose Experiment</h1>
        <h2>Multi-Service Application</h2>
        <p>Web Service: Running</p>
        <p>Redis Service: {status}</p>
        <h3>Total Visits: {visits}</h3>
        <p>Refresh the page to increase the counter.</p>
    </body>
    </html>
    """

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

5.2 requirements.txt

```
Flask
redis
```

5.3 Dockerfile

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]
```

5.4 compose.yaml

```
services:
  web:
    build: .
    ports:
      - "5000:5000"
    environment:
      REDIS_HOST: redis
    depends_on:
      redis:
        condition: service_healthy
    restart: unless-stopped

  redis:
    image: redis:7-alpine
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 5
    restart: unless-stopped
```

6. Procedure

1. Installed and verified Docker Desktop and Docker Compose.
2. Created a project folder named docker-compose-experiment.
3. Created app.py, requirements.txt, Dockerfile, and compose.yaml in the same folder.
4. Added the Flask application code and listed Flask and redis as dependencies.
5. Configured the Dockerfile to install dependencies and run the web application.
6. Configured compose.yaml to define the Flask web service and Redis service.
7. Validated the Compose configuration using docker compose config.
8. Built and started the services using docker compose up --build -d.
9. Checked service status using docker compose ps.
10. Opened http://localhost:5000 in a browser and verified that the web service and Redis connection were displayed.
11. Refreshed the webpage to verify that the visit counter increased.

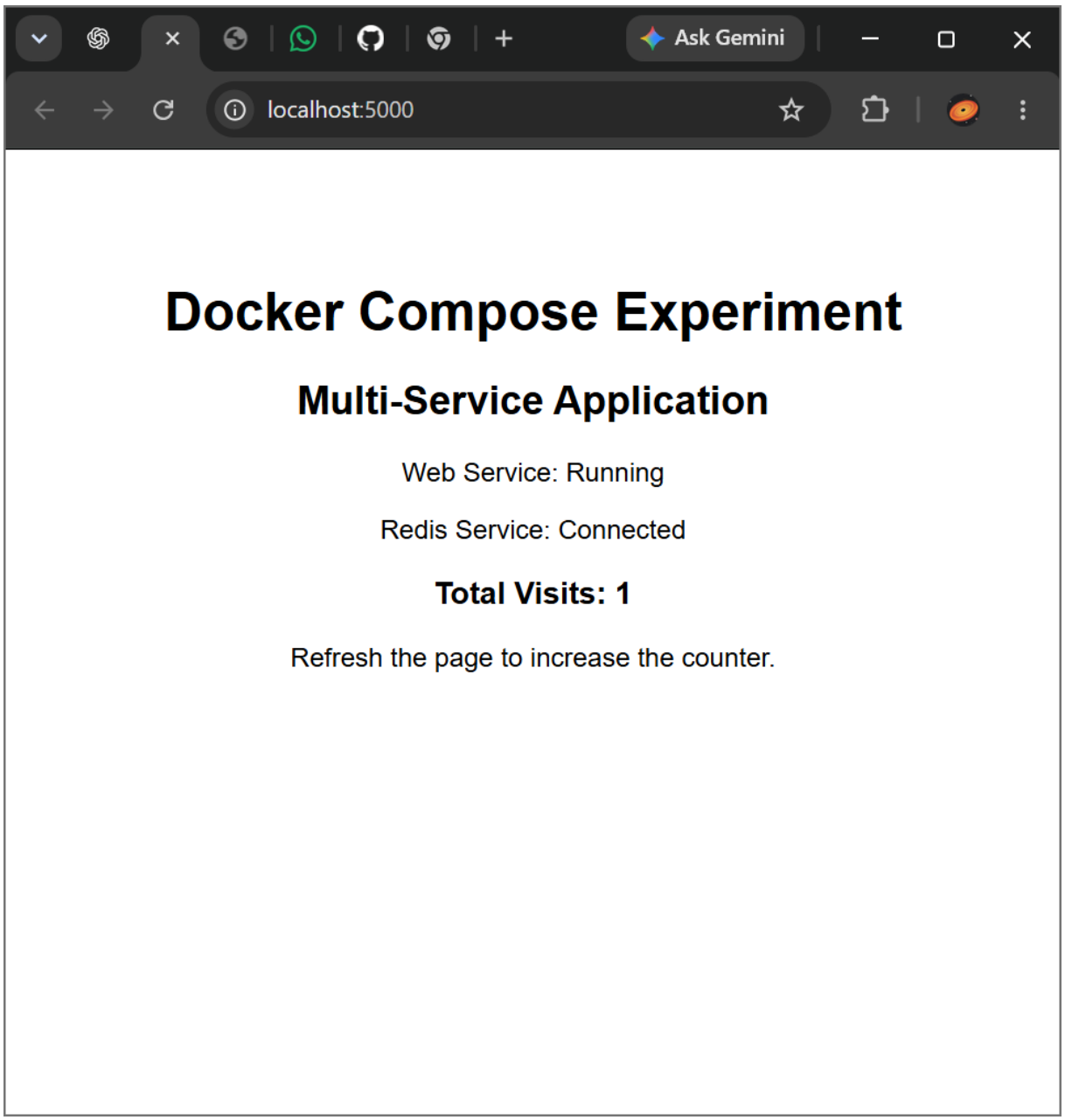
7. Commands Used

Purpose	Command
Check Docker version	docker --version
Check Compose version	docker compose version
Check Docker Engine	docker info
Validate Compose configuration	docker compose config
Build and start services	docker compose up --build -d
List services and status	docker compose ps
View service logs	docker compose logs
Test Redis response	docker compose exec redis redis-cli PING
Stop and remove containers/network	docker compose down

8. Output and Observations

The browser displayed the heading 'Docker Compose Experiment' and 'Multi-Service Application'. The web service status was shown as Running, the Redis service status as Connected, and the visit counter was displayed. The supplied screenshot records Total Visits: 1. Refreshing the webpage should increase the counter on each successful request.

Figure 1: Successfully running multi-service application in the browser.



9. Result Verification

The following commands can be used to verify the deployed services:

```
docker compose ps
docker compose exec redis redis-cli PING
```

Expected Redis response: PONG. The Compose status should list the web and Redis services, with Redis becoming healthy after its health check succeeds.

Note: The browser screenshot confirms the webpage and Redis connection. Capture the output of the verification commands separately if your instructor requires terminal evidence.

10. Conclusion

The experiment to deploy a multi-service application using Docker Compose was performed successfully. The Flask web application and Redis service were defined in a single Compose configuration and run as separate containers. The application was accessed through localhost:5000, and the webpage confirmed that Redis was connected and the visit counter was available. This experiment demonstrates how Docker Compose simplifies the deployment and management of interconnected services.